

517-312 Operating Systems

TERM PROJECT

Mini Job Scheduler & Shared Resource Manager

Java Multithreading: Thread, Synchronization, Semaphore และ Scheduling

อนุญาตให้ใช้ Generative AI ได้ แต่สมาชิกทุกคนต้องรับผิดชอบต่อโค้ดทั้งหมดที่ส่ง และต้องสามารถอธิบาย แก้ไข และทดสอบโค้ดได้ด้วยตนเองในวัน Demo

อาจารย์จะแจกโค้ดตั้งต้น (Starter Code) บางส่วน และไฟล์ชุดงานทดสอบ (workload) แยกจากเอกสารฉบับนี้ เพื่อให้นักศึกษาเริ่มงานได้เร็วขึ้น

1. เป้าหมายของโครงการ

ให้นักศึกษาพัฒนาโปรแกรมจำลองระบบรับงาน (Job) ที่เข้ามาตามเวลา มี Scheduler จัดลำดับงาน และมี Worker Thread หลายตัวรับงานไปทำพร้อมกัน งานบางชนิดต้องใช้ทรัพยากรร่วมที่มีจำนวนจำกัด เช่น Printer และ Database Connection ซึ่งควบคุมด้วย Semaphore

โครงการนี้ไม่ได้จำลอง Kernel หรือ CPU Scheduler จริง และไม่เชื่อมอุปกรณ์หรือฐานข้อมูลจริง

จุดประสงค์คือให้สังเกตและอธิบายพฤติกรรมของ Thread, Queue, Scheduling และ Shared Resource จากโปรแกรมที่รันจริง

เมื่อจบโครงการ นักศึกษาควรสามารถ:

- สร้าง Java Thread หลายตัวและควบคุมให้ทำงานร่วมกันได้ถูกต้อง
- ใช้ BlockingQueue และกลไกป้องกันข้อมูลร่วม (synchronization) โดยไม่วนลูปรอแบบกิน CPU (busy waiting)
- ใช้ Semaphore ควบคุมทรัพยากรที่มีจำนวนสิทธิ์ใช้งานพร้อมกันจำกัด (permit) และคืนสิทธิ์อย่างปลอดภัยเมื่อใช้เสร็จ
- เปรียบเทียบ FCFS กับ Priority Scheduling จาก workload เดียวกัน
- วัด Waiting Time, Turnaround Time, Throughput และ Resource Wait Time จากการรันจริง
- ออกแบบการปิดระบบอย่างถูกต้อง (graceful shutdown) โดยไม่มี Thread ค้างและไม่หยุดก่อน Job สุดท้ายเสร็จ
- อธิบายได้ว่าทำไมโปรแกรมหลาย Thread อาจให้ลำดับเหตุการณ์ต่างกันเล็กน้อยในแต่ละครั้ง แม้ใช้ข้อมูลเดิม (nondeterminism) และแยกสถานะของ Job ออกจาก Java Thread.State ได้

2. สถาปัตยกรรมที่บังคับ

Scheduler ต้องเป็น Thread จริง และต้องทำหน้าที่เชื่อม Arrival Queue กับ Ready Queue/Dispatcher
โครงสร้างขั้นต่ำเป็นดังนี้:

JobGenerator Thread

|

v

arrivalQueue (BlockingQueue)

|

v

Scheduler Thread

|

v

SchedulingPolicy / Ready Queue

(FCFS หรือ Priority)

|

v

Worker Threads (ปรับจำนวนได้)

|

+--> ResourceManager --> Semaphore: PRINTER / DATABASE

|

+--> Statistics + Logger

Monitor Thread --> อ่านสถานะระบบเป็นระยะ

หน้าที่หลัก:

- Main/Controller: สร้าง object, เริ่มระบบ, รอให้จบ, สั่ง shutdown และพิมพ์ผลสรุป
- JobGenerator: ปล่อย Job ตาม arrivalMs ลง arrivalQueue
- Scheduler: take() จาก arrivalQueue แล้วเพิ่ม Job เข้า SchedulingPolicy/Ready Queue
- Worker: รับ Job จาก Ready Queue ไปทำงานพร้อมกันหลายตัว
- ResourceManager: ควบคุม PRINTER และ DATABASE ด้วย Semaphore ที่แชร์ร่วมกันทั้งระบบ

- Monitor: ทุกประมาณ 1,000 ms อ่านจำนวนงานที่รอ/กำลังทำ/เสร็จแล้ว และสถานะ resource โดยต้องอ่านอย่างปลอดภัยแม้หลาย Thread กำลังแก้อัปเดตพร้อมกัน (thread-safe)
- Statistics/Logger: เก็บค่าที่ใช้วัดผล (metrics) และบันทึกเหตุการณ์ (log) ตามรูปแบบที่กำหนด

ข้อกำหนดสำคัญ: ห้ามข้าม Scheduler Thread ด้วยการให้ JobGenerator submit งานตรงเข้า Ready Queue ในเวอร์ชันบังคับ

3. ข้อมูล Job และไฟล์ workload

Field	ตัวอย่าง	ความหมาย
id	J01	รหัส Job ไม่ซ้ำกัน
arrivalMs	300	เวลาที่ Job เข้าระบบ นับจาก simulationStart
priority	1	1 = สูงสุด; ตัวเลขมากขึ้น = priority ต่ำลง
workMs	1200	เวลางานหลักที่จำลองด้วย Thread.sleep(workMs)
resource	PRINTER	NONE / PRINTER / DATABASE
resourceMs	800	เวลาที่ครอบครอง resource จำลองด้วย Thread.sleep(resourceMs)

Job หนึ่งตัวใช้ทรัพยากรร่วม (shared resource) ได้ไม่เกิน 1 ประเภทในส่วนบังคับ เพื่อไม่เพิ่มความซับซ้อนเรื่อง deadlock จากการถือหลาย resource พร้อมกัน

ชุดงานทดสอบ (workload) ต้องอ่านจากไฟล์ภายนอก ห้ามเขียนข้อมูล Job ฝังตายไว้ในโค้ด เพราะวัน Demo อาจารย์อาจใช้ไฟล์งานชุดอื่น

4. Scheduling ที่ต้องรองรับ

Policy	สถานะ	กติกา
FCFS	บังคับ	Job ที่ READY ก่อนถูกเลือกก่อน
Priority	บังคับ	priority = 1 สูงสุด; เมื่อเท่ากันใช้กติกาตัดสินลำดับ (tie-break) จากข้อมูลของ Job ไม่ขึ้นกับจังหวะของ Thread
Aging	Bonus	เพิ่มโอกาสให้ Job priority ต่ำที่รอนาน
MLFQ	Bonus	หลาย queue + time quantum + requeue/remaining work

FCFS และ Priority เป็น non-preemptive ระดับ Job: เมื่อ Worker รับ Job ไปแล้ว Job นั้นทำงานจนไม่ถูกดึงกลับกลางคัน

เมื่อ Priority เท่ากัน กลุ่มต้องกำหนดกติกาตัดสินลำดับ (tie-break) จากข้อมูลของ Job เช่น ลำดับที่เข้ามา, sequence number หรือ id โดยกติกานี้ต้องไม่ขึ้นกับว่า Thread ใดเข้าถึงคิวก่อน ดังนั้น Job

ชุดเดียวกันควรถูกตัดสินตามกติกาเดียวกัน แม้เวลาและลำดับบางบรรทัดใน log อาจต่างกันเล็กน้อยจากการสลับ Thread ของ Java/OS

5. Shared Resource และการจำลองเวลา

Resource	ค่าเริ่มต้น	ความหมาย
PRINTER	1 permit	มี Printer เชิงตรรกะที่ใช้งานพร้อมกันได้ 1 Job
DATABASE	2 permits	มี Database connections เชิงตรรกะพร้อมกันได้ 2 Job
NONE	-	Job ไม่ขอ shared resource

จำนวนสิทธิ์ให้ใช้ resource พร้อมกัน (permit) ต้องปรับได้จาก command line โดย Worker ทุกตัวใช้

ResourceManager object เดียวกัน และในส่วนบังคับให้สร้าง Semaphore แบบ fair=true

เพื่อให้ลำดับการรอมีความเป็นธรรมและผลของแต่ละกลุ่มเทียบกันได้ง่ายขึ้น

ใช้ Thread.sleep(workMs) และ Thread.sleep(resourceMs) เป็นการจำลองเวลาอย่างตั้งใจ ไม่ใช่การวัด CPU burst หรือ I/O จริง การใช้ sleep ช่วยควบคุมเวลาพื้นฐานของ workload ให้ใกล้เคียงกันข้ามเครื่อง

และทำให้ผลของการเปลี่ยนจำนวน permit มีความได้ตรงกับตัวแปรที่ทดลอง

ทุกจุดที่ขอใช้ resource (acquire) ต้องรับประกันว่าจะคืนสิทธิ์ (release) แม้เกิดข้อผิดพลาดหรือ Thread ถูก interrupt เช่นใช้ try/finally หรือวิธีที่ปลอดภัยเทียบเท่า

ห้ามใช้ Thread.stop(), Thread.suspend(), Thread.resume(), busy waiting และห้ามใช้ sleep เพื่อ “เดา” ว่า Thread อื่นน่าจะทำงานเสร็จแล้ว

6. ลำดับการทำงานของ Worker

1. Worker รับ Job และบันทึก startTime
2. จำลองงานหลักด้วย Thread.sleep(workMs)
3. ถ้า resource != NONE ให้บันทึกเวลาเริ่มรอ resource แล้ว acquire Semaphore

- เมื่อ acquire สำเร็จ ให้สะสม Resource Wait Time
- จำลองช่วงที่ครอบครอง resource ด้วย Thread.sleep(resourceMs)
- release resource อย่างปลอดภัย
- บันทึก completionTime และอัปเดต Statistics โดยต้องปลอดภัยเมื่อ Worker หลายตัวแก้ไขข้อมูลพร้อมกัน (thread-safe)

สถานะ ARRIVED / READY / RUNNING / WAITING_RESOURCE / COMPLETED เป็น Job state ของโปรเจกต์ ไม่จำเป็นต้องตรงกับ Java Thread.State แบบหนึ่งต่อหนึ่ง

7. การป้องกันข้อมูลร่วมและการปิดระบบ (Synchronization & Graceful Shutdown)

Queue หรือตัวแปรที่หลาย Thread ใช้ร่วมกันต้องถูกป้องกันไม่ให้เกิดการอ่าน/เขียนชนกัน ตัวนับและ Statistics ที่หลาย Worker แก้พร้อมกันต้องใช้วิธีที่เหมาะสม เช่น synchronized, Lock หรือ Atomic classes

runningJobs เป็นข้อมูลร่วมที่ Monitor อ่านในขณะที่ Worker เพิ่ม/ลดค่าอยู่ จึงต้องป้องกันให้หลาย Thread ใช้ร่วมกันได้อย่างปลอดภัย เช่นเดียวกับตัวนับ ready และ completed

ปัญหาการปิดระบบที่ทุกกลุ่มต้องแก้: เมื่อ JobGenerator ส่งงานครบแล้ว Worker ที่กำลังรออยู่ใน takeNextJob() จะไม่รู้เองว่าจะไม่มี Job ใหม่เข้ามาอีก กลุ่มต้องออกแบบวิธีบอกให้ระบบรู้ว่า “ส่งงานครบแล้ว” และ “งานทั้งหมดเสร็จแล้ว” จากนั้นจึงหยุด Worker/Monitor ทุกตัวโดยไม่ทิ้งงานค้าง

แนวทางที่สามารถค้นคว้าเพิ่มเติมได้ เช่น Poison Pill (ส่งค่าพิเศษเพื่อบอก Worker ให้หยุด), CountdownLatch (ตัวช่วยรอให้งานครบตามจำนวน) หรือการนับจำนวน Job ที่ยังไม่เสร็จโดยป้องกันตัวนับให้ปลอดภัยเมื่อหลาย Thread ใช้ร่วมกัน เอกสารนี้ไม่บังคับว่าต้องใช้วิธีใด

ระบบต้องไม่เกิดอาการต่อไปนี้:

- main() จบแล้วแต่ JVM ยังไม่ยุติเพราะมี Thread ค้าง
- Worker ถูกหยุดก่อน Job สุดท้ายเสร็จ
- permit ของ Semaphore สูญหายหรือค้างเมื่อเกิด InterruptedException/exception ระหว่างถือ resource
- ใช้ System.exit() เพื่อหลบปัญหา shutdown แทนการแก้สาเหตุ

8. ค่าที่ใช้วัดผลและบันทึกเหตุการณ์ (Metrics & Logging)

Metric	นิยาม
Waiting Time	startTime - actualArrivalTime
Turnaround Time	completionTime - actualArrivalTime

Metric	นิยาม
Resource Wait Time	เวลาตั้งแต่เริ่มรอ Semaphore จน acquire สำเร็จ; resource=NONE ให้ค่า 0
Throughput	จำนวน Job ที่เสร็จ / เวลารวมของ simulation (jobs/second)

รายงานค่าเฉลี่ย Waiting Time และ Turnaround Time จาก Job ทุกตัว ส่วน average Resource Wait Time ให้คำนวณเฉพาะ Job ที่ขอ PRINTER หรือ DATABASE

สมการตรวจสอบความถูกต้องของค่าแต่ละ Job:

Turnaround Time = Waiting Time + workMs + Resource Wait Time + resourceMs

สำหรับ resource=NONE ให้ Resource Wait Time = 0 และ resourceMs = 0 สมการนี้ใช้ตรวจสอบ Job

ไม่ให้นำค่าเฉลี่ยแต่ละคอลัมน์มาบวกกันโดยตรง

อาจารย์จะแจกตัวบันทึก log ตั้งต้นให้ โดยกำหนดรูปแบบเดียวกันทุกกลุ่ม เวลาใน log ให้นับจาก simulationStart และทุกบรรทัดต้องมีชื่อ Thread เช่น ARRIVED, START, WAIT, ACQUIRE, RELEASE, COMPLETE และสถานะจาก Monitor

[0000 ms] [JobGenerator] J01 ARRIVED priority=5

[0004 ms] [Scheduler] J01 READY

[0012 ms] [Worker-1] J01 START

[2420 ms] [Worker-2] J02 WAIT PRINTER

[2421 ms] [Worker-2] J02 ACQUIRE PRINTER

[3622 ms] [Worker-2] J02 RELEASE PRINTER

[3623 ms] [Worker-2] J02 COMPLETE

แม้ใช้ workload เดิม ผลการรันแต่ละครั้งอาจต่างกันเล็กน้อยที่เวลาและลำดับบางบรรทัดใน log

เพราะระบบปฏิบัติการอาจสลับลำดับการทำงานของ Thread ไม่เหมือนเดิมทุกครั้ง

แต่กติกาของโปรแกรมและแนวโน้มของผลต้องอธิบายได้

9. รูปแบบการรันที่กำหนด

java Main <workload.csv> <fcfs|priority> <workers> <printerPermits> <databasePermits>

ตัวอย่าง:

java Main jobs_standard.csv priority 3 1 2

โปรแกรมต้องรับค่าจาก command line ให้ครบทั้ง 5 ตัว หากใส่ไม่ครบหรือค่าผิด ให้แสดงวิธีใช้ (Usage)

ที่อ่านเข้าใจได้แล้วจบก่อนเริ่ม Thread ใด ๆ ห้ามแอบใช้ค่า default โดยไม่บอก

เพราะผู้ตรวจต้องรู้ซัวกำลังรันด้วยค่าใด

Usage: java Main <workload.csv> <fcfs|priority> <workers> <printerPermits> <databasePermits>

WorkloadLoader.java จะถูกแจกให้ใช้สำหรับอ่านและตรวจสอบความถูกต้องของไฟล์ CSV นักศึกษาไม่ต้องเขียนตัวอ่าน CSV เอง เพราะไม่ใช่สิ่งที่โครงงานนี้ต้องการวัด

10. Monitor Thread

Monitor เป็น Thread บังคับ ให้รายงานสถานะระบบประมาณทุก 1,000 ms (ไม่ต้องแม่นยำตรง 1,000 ms ทุกครั้ง)

อย่างน้อยควรมี ready, running, completed และสถานะการใช้ resource

ข้อมูลที่ Monitor อ่านต้องเป็น snapshot (ภาพรวมสถานะระบบ ณ ขณะหนึ่ง) ที่อ่านได้อย่างปลอดภัย โดยเฉพาะ runningJobs ซึ่ง Worker หลายตัวอาจเพิ่ม/ลดพร้อมกัน ห้ามอ่าน collection

หรือตัวนับที่กำลังถูกแก้ไขโดยไม่มีการป้องกันที่เหมาะสม

11. Workload ที่แจกให้

ไฟล์	วัตถุประสงค์
jobs_standard.csv	ใช้ทดสอบ Scheduling และจำนวน Worker; ออกแบบให้ FCFS/Priority ให้ผลต่างชัดและมีการแย่งใช้ resource บางส่วน
jobs_printer.csv	ทำให้ Printer เป็นจุดคอขวดชัดเจน สำหรับทดสอบ permit 1 เทียบ 2
jobs_db.csv	ตรวจสอบว่า DATABASE permits=2 อนุญาตพร้อมกันได้สูงสุด 2 Job
jobs_same_priority.csv	ตรวจสอบว่าทิกาดัดสินลำดับทำงานถูกเมื่อ priority เท่ากันทั้งหมด
jobs_single.csv	กรณีทดสอบง่ายที่สุด: มี Job เดียว และระบบต้องปิดตัวเองได้ถูกต้อง

อาจารย์อาจใช้ workload อื่นในวัน Demo จึงห้ามผูก logic กับ id หรือจำนวน Job ของไฟล์ตัวอย่าง

12. การทดลองที่ต้องทำ

ใช้โค้ดชุดเดียวกันในการทดลองทั้งหมด เปลี่ยนเฉพาะค่าที่ใช้รันและไฟล์ workload ตามที่กำหนด ห้ามแก้ algorithm ให้แต่ละรอบทำงานคนละแบบ

การทดลอง	ชุดค่าที่ใช้รัน	สิ่งที่ต้องสังเกต
A. Scheduling	jobs_standard.csv; workers=3; printer=1; database=2; เทียบ FCFS กับ Priority	avg WT, avg TAT, ลำดับเริ่มงาน และ Job ใดได้/เสียประโยชน์
B. Worker count	jobs_standard.csv; Priority; printer=1; database=2; workers=1, 3, 5	Throughput, avg Resource Wait และจุดที่เพิ่ม Worker ต่อแล้วได้ประโยชน์น้อยลง
C. จุดคอขวดของ Resource	jobs_printer.csv; Priority; workers=3; database=2; printer=1 vs 2	avg Resource Wait, Throughput และดูว่าจุดคอขวดของระบบเปลี่ยนอย่างไร

เพื่อดูว่าทำไมการรันซ้ำจึงอาจได้เวลาไม่ตรงกันทุก millisecond ให้รันชุดอ้างอิงเดิมซ้ำอีก 1 รอบโดยไม่เปลี่ยนค่าใด:

jobs_standard.csv, Priority, workers=3, printer=1, database=2 แล้วเปรียบเทียบกับรอบแรก

13. สิ่งที่ต้องส่ง

- โค้ดทั้งหมด (.java)
 - README.md สั้น ๆ: วิธี compile/run, ค่าที่ใส่ตอนรันโปรแกรม (command-line arguments) และข้อจำกัดที่ควรรู้
 - ไฟล์ log ดิบจากแต่ละชุดการรันที่กำหนด (สามารถ redirect ลงไฟล์ได้ ไม่ต้องจัดหน้าเป็นรายงาน)
 - ผลลัพธ์ 1 หน้า: ตารางสรุปผล + คำตอบ 4 ข้อในหัวข้อ 14
 - บันทึกการใช้ AI: ระบุว่าใช้เครื่องมือใด ช่วยเรื่องใด และอย่างน้อย 1 จุดที่กลุ่มต้องแก้หรือตรวจสอบคำตอบของ AI เอง
- ไม่ต้องส่งรายงาน 5-8 หน้า, ไม่ต้องส่งแผนผังโครงสร้างระบบเป็นไฟล์, ไม่ต้องส่งภาพหน้าจอ และไม่ต้องส่ง .class / IDE cache / target/build directory

14. รูปแบบผลลัพธ์ 1 หน้า

ตารางสามารถจัดรูปแบบเองได้ แต่ต้องมีอย่างน้อย 7 แถวต่อไปนี้ โดยกรอกค่าจากการรันจริง:

#	Workload / Policy	Workers	Printer	DB	avg WT	avg TAT	Throughput	avg RW
1	standard / FCFS	3	1	2				
2	standard / Priority	3	1	2				

#	Workload / Policy	Workers	Printer	DB	avg WT	avg TAT	Throughput	avg RW
3	standard / Priority	1	1	2				
4	standard / Priority	5	1	2				
5	printer / Priority	3	1	2				
6	printer / Priority	3	2	2				
7	standard / Priority (repeat)	3	1	2				

ให้รายงาน avg WT, avg TAT และ avg Resource Wait เป็นจำนวนเต็มหน่วย milliseconds (ms) และรายงาน Throughput อย่างน้อย 2 ตำแหน่งทศนิยม สำหรับแถว 2 และ 7 ให้เปรียบเทียบ timestamp และลำดับเหตุการณ์ใน Raw Log ประกอบ ไม่ใช่ดูเฉพาะค่าเฉลี่ย

ตอบคำถามต่อไปนี้ข้อละประมาณ 2-3 บรรทัด:

1. เพิ่ม Worker จาก 3 เป็น 5 แล้ว Throughput เพิ่มขึ้นสัดส่วนหรือไม่ และ avg Resource Wait เปลี่ยนอย่างไร เพราะอะไร?
2. FCFS กับ Priority ทำให้ Job กลุ่มใดได้ประโยชน์หรือเสียประโยชน์จาก jobs_standard.csv? ให้อ้างอิง Waiting Time และลำดับ START ใน log และอธิบายด้วยว่าเหตุใด Priority Scheduling ในการทดลองนี้อาจมี Throughput ต่ำกว่า FCFS แม้ค่าเฉลี่ย Waiting Time จะดีกว่า
3. เมื่อเพิ่ม Printer permit จาก 1 เป็น 2 แล้ว avg Resource Wait และ Throughput เปลี่ยนอย่างไร และจุดคอขวดของระบบย้ายไปอยู่ที่ใด?
4. เปรียบเทียบแถว 2 กับแถว 7: เหตุใด workload และชุดค่าที่ใช้รันเหมือนเดิม จึงอาจให้เวลาใน log ลำดับเหตุการณ์ และค่าที่วัดต่างกันเล็กน้อย?

15. Demo และการประเมินรายบุคคล

สมาชิกทุกคนต้องเข้าร่วม Demo และสามารถอธิบายโค้ดทั้งระบบ ไม่ใช่เฉพาะไฟล์ที่ตนเองเขียน ระหว่าง Demo อาจารย์อาจให้วาดโครงสร้างระบบบนกระดาน ชี้ข้อมูลที่หลาย Thread ใช้ร่วมกัน เปลี่ยนค่าที่ใช้รัน หรือให้แก้เงื่อนไขเล็กน้อยต่อหน้า

ตัวอย่างหัวข้อที่อาจถูกถาม/ทดสอบ:

- เปลี่ยนจำนวน Worker หรือจำนวนสิทธิ์ของ Semaphore แล้วอธิบายว่าคาดว่าระบบจะเปลี่ยนอย่างไร

- ชี้ข้อมูลรวมที่หลาย Thread แก้ไขได้ เช่น runningJobs และอธิบายว่าถ้าเอาการป้องกันออกจะเกิดปัญหาอะไร
- อธิบาย FCFS/Priority และกติกาตัดสินลำดับ (tie-break) จากโค้ดจริง
- อธิบายวิธีปีติระบบของกลุ่ม และตอบได้ว่าถ้าเพิ่ม Worker ระหว่างทำงาน วิธีนั้นยังใช้ได้ถูกต้องหรือไม่
- อธิบายว่าถ้า Worker ถูก interrupt หลัง acquire resource จะป้องกัน permit หายอย่างไร
- เปรียบเทียบ Thread.State ตอนรอ Semaphore กับตอน Thread.sleep(resourceMs)
- ใช้สมการ $TAT = WT + workMs + Resource\ Wait + resourceMs$ ชี้ว่าถ้าค่าไม่ลง timestamp ใดน่าจะผิด
- อธิบายว่า Generative AI ช่วยสร้างส่วนใด และกลุ่มตรวจสอบความถูกต้องของส่วนนั้นอย่างไร

16. นโยบายการใช้ Generative AI

อนุญาตให้ใช้ Generative AI เพื่อช่วยวางโครงสร้าง อธิบาย API ตรวจสอบข้อผิดพลาด สร้างชุดทดสอบ และสร้างโค้ดบางส่วนได้ แต่สมาชิกในกลุ่มต้องรับผิดชอบต่อโค้ดทั้งหมดที่ส่ง และต้องสามารถอธิบาย แก้ไข และทดสอบได้ด้วยตนเอง

การที่โปรแกรมรันได้ไม่เพียงพอสำหรับคะแนนเต็ม สมาชิกต้องอธิบายได้ว่า Thread แต่ละตัวทำอะไร ข้อมูลใดใช้ร่วมกัน ป้องกันการชนกันอย่างไร ใช้ Semaphore/Scheduling อย่างไร ค่าที่วัดหมายถึงอะไร และปีติระบบอย่างไร คะแนน Demo จะสะท้อนความเข้าใจของแต่ละคน

17. เกณฑ์การให้คะแนน (100 คะแนน)

หัวข้อ	คะแนน	เกณฑ์หลัก
Demo + ความเข้าใจรายบุคคล (Demo + Individual Understanding)	35	อธิบายโครงสร้างระบบและโค้ดของกลุ่มได้ แก่ใจทฤษฎีได้ดี และตอบคำถามรายบุคคล
การออกแบบ Thread และการทำงานพร้อมกัน (Thread & Concurrency Design)	15	โครงสร้าง Thread ชัด ไม่วนลูปรอแบบกิน CPU และประสานงานระหว่าง Thread ได้เหมาะสม
การป้องกันข้อมูลรวมและ Resource (Synchronization & Resource)	15	ป้องกัน race condition ได้ ใช้ Semaphore ถูกต้อง และคืน resource ได้ปลอดภัยเมื่อเกิดข้อผิดพลาด
ความครบถ้วนของโปรแกรม (Functional Requirements)	15	อ่าน workload, หลาย Worker, Monitor, ทำงานครบ, shutdown ถูกต้อง, README ใช้งานได้

หัวข้อ	คะแนน	เกณฑ์หลัก
การจัดลำดับงาน (Scheduling)	10	FCFS + Priority ถูกต้อง และกติกาคัดสัณลำดับ (tie-break) ชัดเจน
การทดลองและการอธิบายผล (Experiments & Analysis)	10	ตารางผลครบ รันตามชุดค่าที่กำหนด และอธิบายความหมายของผลทั้ง 4 คำถามได้

18. จุดตรวจกลางทางและชุดทดสอบ (Checkpoint & Test Cases)

จุดตรวจกลางทาง (กำหนดวันโดยอาจารย์): ต้องสาธิตให้เห็นอย่างน้อยว่า Job เข้าระบบผ่าน JobGenerator -> Scheduler และมี Worker หลายตัวดึงงานจาก Ready Queue เดียวกันได้ ยังไม่จำเป็นต้องมี resource หรือค่าที่วัดครบ ก่อนส่งจริง ทุกกลุ่มควรทดสอบอย่างน้อย:

- jobs_single.csv: มี Job เดียว ต้องทำงานและ shutdown โดยไม่ค้าง
- jobs_standard.csv: เห็น Worker หลายตัวทำงานพร้อมกันและ FCFS/Priority ให้ลำดับ START ต่างกัน
- jobs_printer.csv: printerPermits=1 ต้องไม่มี 2 Job ครอบครอง PRINTER พร้อมกัน
- jobs_db.csv: databasePermits=2 ต้องอนุญาตให้สูงสุด 2 Job ใช้ DATABASE พร้อมกัน
- jobs_same_priority.csv: กติกาคัดสัณลำดับ (tie-break) ต้องทำงานตามกติกาที่กลุ่มกำหนด
- interrupt Worker ระหว่างรอ resource: โปรแกรมต้องไม่ค้างและไม่ทำ permit หาย
- หลัง print summary แล้ว JVM ต้องปิดได้เองโดยไม่มี Worker/Monitor ค้าง

19. งานเสริม (Bonus / Extension)

งานเสริมไม่ใช่ส่วนบังคับ และไม่ควรทำก่อนส่วนหลักของโครงงานทำงานครบ:

- Aging เพื่อลด starvation ของ Priority Scheduling
- MLFQ ที่มี time quantum, remainingWorkMs และ requeue
- Dynamic Worker เพิ่ม/ลดจำนวน Worker ตาม queue length
- Job cancellation หรือ resource timeout ด้วย tryAcquire()
- Virtual Thread comparison สำหรับ workload ที่มีการรอจำนวนมาก
- Deadlock challenge: Job ใช้ resource มากกว่า 1 ชนิดและออกแบบป้องกัน deadlock

ภาคผนวก A: Workload ที่แจก

ไฟล์จริงจะแจกแยกต่างหาก นักศึกษาไม่ต้องคัดลอกจากเอกสารนี้ รายการด้านล่างใช้เพื่ออ้างอิง

A.1 jobs_standard.csv

id	arrivalMs	priority	workMs	resource	resourceMs
J01	0	5	2500	NONE	0
J02	100	5	2400	PRINTER	1200
J03	200	5	2500	NONE	0
J04	300	5	2400	DATABASE	1000
J05	400	5	2500	PRINTER	1200
J06	500	4	2400	DATABASE	1000
J07	1000	1	700	PRINTER	900
J08	1200	1	600	PRINTER	900
J09	1400	1	700	DATABASE	900
J10	1600	2	600	NONE	0

A.2 jobs_printer.csv

id	arrivalMs	priority	workMs	resource	resourceMs
P01	0	2	250	PRINTER	1200
P02	100	1	300	PRINTER	1200
P03	200	3	250	PRINTER	1200
P04	300	2	300	PRINTER	1200
P05	400	1	250	PRINTER	1200
P06	500	3	300	PRINTER	1200

A.3 jobs_db.csv

id	arrivalMs	priority	workMs	resource	resourceMs
D01	0	2	250	DATABASE	1200
D02	100	1	300	DATABASE	1200
D03	200	3	250	DATABASE	1200
D04	300	2	300	DATABASE	1200
D05	400	1	250	DATABASE	1200

D06,500,3,300,DATABASE,1200

A.4 jobs_same_priority.csv

id,arrivalMs,priority,workMs,resource,resourceMs

S01,0,2,900,NONE,0

S02,100,2,900,NONE,0

S03,200,2,900,NONE,0

S04,300,2,900,NONE,0

S05,400,2,900,NONE,0

S06,500,2,900,NONE,0

A.5 jobs_single.csv

id,arrivalMs,priority,workMs,resource,resourceMs

ONE01,0,2,800,NONE,0

ภาคผนวก B: โค้ดเริ่มต้น (Starter Code) และขอบเขตที่ยังต้องออกแบบเอง

โค้ดเริ่มต้นจะให้โค้ดพื้นฐานที่จำเป็นต่อการเริ่มโปรเจกต์ แต่ไม่ใช่สาระหลักที่ต้องการวัด เช่น WorkloadLoader.java

สำหรับอ่าน CSV และ ProjectLogger.java สำหรับบันทึก log รวมถึง class/interface พื้นฐานตามที่อาจารย์แจก

WorkloadLoader รับผิดชอบการอ่านและตรวจสอบความถูกต้องของ CSV อยู่แล้ว จึงไม่มีการให้คะแนนเรื่องการเขียนตัวอ่าน CSV หรือการตรวจไฟล์ CSV ที่รูปแบบผิด

ส่วนที่นักศึกษาจะต้องออกแบบและเขียนเอง:

- โครงสร้างคิวพร้อมทำงาน (Ready Queue) สำหรับ FCFS/Priority และ Scheduler Thread
- กฎการตัดสินลำดับ (tie-break) เมื่อ Priority เท่ากัน
- วงจรการทำงานของ Worker ตั้งแต่เริ่ม รับงาน ทำงาน จนหยุด และ method processJob()
- ResourceManager และการใช้ Semaphore รวมถึงการคืน resource อย่างปลอดภัยเมื่อเกิดข้อผิดพลาด
- การเก็บค่าที่ใช้วัดผลของแต่ละ Job และ Statistics ที่หลาย Worker อัปเดตพร้อมกันได้อย่างปลอดภัย
- ตัวนับ runningJobs/ready/completed ที่ Monitor อ่านได้อย่างปลอดภัยขณะที่ Worker กำลังแก้ค่า
- วิธีบอกว่า “งานทั้งหมดเสร็จแล้ว” และวิธีหยุด Worker/Monitor อย่างถูกต้อง
- การจัดการเมื่อ Thread ถูก interrupt ระหว่างรอหรือใช้ resource
- การทดสอบเพิ่มเติม และเหตุผลในการออกแบบที่ใช้ตอบคำถามใน Demo

โค้ดตั้งต้นมีไว้ช่วยลดงานพื้นฐานที่ไม่ใช่สาระหลัก เช่น การอ่านไฟล์และรูปแบบ log

แต่ส่วนที่เกี่ยวกับการทำงานพร้อมกันของหลาย Thread เช่น queue, scheduler, worker, synchronization, semaphore, metrics และการปิดระบบ นักศึกษายังต้องออกแบบและทำเอง